

Prova 2

1. Annotations.....	1
1.1 O que são anotações (annotations) em Java?.....	1
1.2 Exemplos.....	2
1.2.1 @OneToMany.....	2
1.2.2 mappedBy = "pessoa".....	2
1.2.3 @ManyToMany.....	3
1.2.4 @JoinTable.....	3
1.2.5 @JoinColumn.....	4
1.2.6 @OneToOne.....	4
1.2.7 cascade = CascadeType.ALL (ou PERSIST, REMOVE, etc).....	4
1.2.8 orphanRemoval = true.....	5
1.3 Lado Dono vs Lado Inverso.....	6
1.4 Diferença entre @JoinColumn e @JoinTable.....	6
1.4.1 @JoinColumn.....	6
1.4.2 @JoinTable.....	7
1.5 Como modelar N:N sem tabela extra visível.....	8
1.5.1 Usando @ManyToMany (JPA gerencia a tabela, você não vê).....	8
1.5.2 Transformar N:N em duas N:1 criando a entidade de relação.....	8
1.5.3 Transformar N:N em 1:N + N:1 usando mappedBy.....	9
1.6 Quando usar CascadeType.....	9
1.6.1 Regra Geral.....	10
2. Boas práticas de relacionamentos JPA.....	10
3. Consultas JPQL.....	10
3.1 Consultas Básicas.....	10
3.2 Consultas Avançadas (JOIN, agregação, relacionamentos).....	11
3.3 Consultas para evitar exclusão indevida (verificação de dependência).....	12
3.4 Dicas.....	12

1. Annotations

1.1 O que são anotações (*annotations*) em Java?

Anotações são **metadados**, ou seja, informações adicionais que você coloca no código para instruir o compilador, o framework ou o runtime sobre como aquela classe/campo/método deve se comportar.

Elas **não têm lógica** por si mesmas — quem interpreta é o framework. No caso do JPA, as anotações dizem ao Hibernate (ou EclipseLink etc) como mapear as classes Java para tabelas no banco.

Exemplos simples de onde você já viu anotações:

Anotação	Usada para
<code>@Override</code>	Diz que um método sobrescrever outro
<code>@Entity</code>	Diz que a classe representa uma tabela
<code>@Id</code>	Define a chave primária
<code>@GeneratedValue</code>	Diz como a PK é gerada

Você nunca chama uma anotação, você só **declara** e o framework interpreta.

1.2 Exemplos

1.2.1 `@OneToMany`

Significa: **uma Pessoa tem muitas Vendas**, por exemplo.

```
@OneToMany(mappedBy = "pessoa")  
  
private List<Venda> vendas;
```

- Representa relação **1:N** (um para muitos)
- Esse lado da relação é o lado **inverso** (não contém a FK)
- `mappedBy` indica qual atributo, na outra entidade, é o dono da relação

1.2.2 `mappedBy = "pessoa"`

Essa parte diz: "*o relacionamento é controlado pelo atributo `pessoa` na classe `Venda`*".

Ou seja, em `Venda` deve existir algo assim:

```
@ManyToOne  
  
@JoinColumn(name = "pessoa_id")
```

```
private Pessoa pessoa;
```

- Evita criar tabela extra desnecessária
- Evita inconsistências

1.2.3 @ManyToMany

Relação N:N (muitos para muitos)

```
@ManyToMany(cascade = CascadeType.PERSIST)
```

```
private List<Veiculo> veiculos;
```

- Exemplo real: “Pessoa pode ter vários veículos” e “Veículo pode pertencer a várias pessoas”
- Essa relação gera uma **tabela intermediária**
- Pode ser bidirecional ou unidirecional

1.2.4 @JoinTable

Usado **apenas em @ManyToMany** para especificar a *tabela que faz o meio de campo* entre as duas entidades.

```
@JoinTable(  
    name = "pessoa_veiculo",  
    joinColumns = @JoinColumn(name = "pessoa_codigo"),  
    inverseJoinColumns = @JoinColumn(name = "veiculo_codigo")  
)
```

Isso cria uma tabela assim:

pessoa_veiculo

pessoa_codigo

veiculo_codigo

1.2.5 @JoinColumn

Define qual **coluna** no **banco** é a **chave estrangeira** que liga as tabelas.

Exemplo:

```
@JoinColumn(name = "endereco_codigo", nullable = false)
```

Isso significa que a tabela **pessoa** terá uma coluna **endereco_codigo** que referencia a PK da tabela **endereco**.

1.2.6 @OneToOne

Relação **1:1** (uma pessoa tem um endereço único)

```
@OneToOne(cascade = CascadeType.ALL)

@JoinColumn(name = "endereco_codigo", nullable = false)

private Endereco endereco;
```

- Apenas uma entidade pode ter aquele endereço
- **@JoinColumn** indica que a FK está na tabela **pessoa**
- Pode usar **mappedBy** se a FK estiver no outro lado

1.2.7 cascade = CascadeType.ALL (ou PERSIST, REMOVE, etc)

Define se ações feitas na entidade **devem refletir** nas entidades relacionadas.

Exemplo:

```
@OneToOne(cascade = CascadeType.ALL)

private Endereco endereco;
```

Isso significa:

Ação em Pessoa

Acontece também em Endereco?

`persist(pessoa)`

✓ gravar o endereço junto

`remove(pessoa)`

✓ remover o endereço

`merge(pessoa)`

✓ atualizar endereço

- `CascadeType.PERSIST` → só propaga `persist`
 - Se você salvar (`entityManager.persist(parent)`) a entidade **pai**, as entidades **filhas** que estiverem associadas e ainda não tiverem ID também serão persistidas automaticamente.
 - Não propaga `merge`, `remove`, `refresh`, `detach` ou `all`.
- `CascadeType.REMOVE` → só remove
- `CascadeType.ALL` → tudo (`persist`, `merge`, `remove`, `refresh`)

1.2.8 `orphanRemoval = true`

Serve para **remover automaticamente objetos que perderam o vínculo com a entidade principal**.

```
@OneToMany(mappedBy = "pessoa", orphanRemoval = true)

private List<Venda> vendas;
```

Exemplo real:

```
pessoa.getVendas().remove(venda);
```

Se `orphanRemoval = true`, então a venda é removida do banco automaticamente.

Sem isso, ficaria órfã na tabela.

- Muito usado em listas de endereços, telefones, itens de pedido, etc.

1.3 Lado Dono vs Lado Inverso

Em relacionamentos bidirecionais, **somente um lado comanda a FK no banco.**

Lado	Quem tem o <code>@JoinColumn</code>
✓ Dono	Sim
✗ Inverso	Usa <code>mappedBy</code>

Exemplo:

```
// DONO

@ManyToOne

@JoinColumn(name = "pessoa_id")

private Pessoa pessoa;
```

```
// INVERSO

@OneToMany(mappedBy = "pessoa")

private List<Venda> vendas;
```

- Alterar apenas o lado inverso **não atualiza o banco.**
- Você deve atualizar o **lado dono da relação.**

1.4 Diferença entre `@JoinColumn` e `@JoinTable`

1.4.1 `@JoinColumn`

Usado quando existe chave estrangeira direta na tabela

Usado em relacionamentos OneToOne ou ManyToOne, onde a FK fica na própria tabela da entidade.

- **Exemplo:** Cada Veículo pertence a uma Pessoa.

```
@Entity

public class Veiculo {

    @ManyToOne

    @JoinColumn(name = "pessoa_id") // FK fica em VEICULO

    private Pessoa pessoa;

}
```

Isso gera uma tabela assim:

```
VEICULO

| id | nome | pessoa_id |
```

- Usa **apenas 2 tabelas**
- Só funciona para relações 1:N ou 1:1

1.4.2 @JoinTable

Usado quando existe tabela intermediária

Usado em **ManyToMany**, ou quando você quer controlar manualmente a tabela de junção.

- **Exemplo:** Pessoa pode ter vários Veículos e um Veículo pode ter várias Pessoas.

```
@Entity

public class Pessoa {

    @ManyToMany

    @JoinTable(

        name = "pessoa_veiculo",

        joinColumns = @JoinColumn(name = "pessoa_id"),

        inverseJoinColumns = @JoinColumn(name = "veiculo_id")

    )
```

```

        private List<Veiculo> veiculos;

    }

```

Tabela gerada:

```

PESSOA

VEICULO

PESSOA_VEICULO (tabela de junção)

| pessoa_id | veiculo_id |

```

- Necessário para N:N;
- Pode ter colunas extras (ex: data de compra);
- Sempre cria uma tabela extra.

1.5 Como modelar N:N sem tabela extra visível

- *Tecnicamente, em banco relacional sempre existe uma tabela intermediária.*
- A diferença é: **ela pode ser escondida e manipulada pelo JPA.**

Três formas de fazer isso:

1.5.1 Usando @ManyToMany (JPA gerencia a tabela, você não vê)

```

@ManyToMany
private List<Veiculo> veiculos;

```

- Resultado: JPA cria `pessoa_veiculo`, mas você não precisa criar a entidade da tabela.
- Porém: **não é possível adicionar mais atributos na relação**, ex: "data que a pessoa comprou o veículo".

1.5.2 Transformar N:N em duas N:1 criando a entidade de relação

- Exemplo: Pessoa compra Veículo → precisa registrar a data da compra.

Cria uma nova classe:

```

@Entity

public class PessoaVeiculo {

    @ManyToOne

```

```

        private Pessoa pessoa;

        @ManyToOne

        private Veiculo veiculo;

        private LocalDate dataCompra;

    }

```

Agora o relacionamento vira assim:

```

Pessoa 1 --- N PessoaVeiculo N --- 1 Veiculo

```

- Permite atributos extras
- Evita problemas do @ManyToMany
- Mais uso profissional

1.5.3 Transformar N:N em 1:N + N:1 usando **mappedBy**

Útil quando a relação não é realmente N:N.

1.6 Quando usar **CascadeType**

Cascade	Propaga?
PERSIST	salvar
REMOVE	excluir
MERGE	atualizar
REFRESH	recarregar
DETACH	remover do contexto
ALL	todos acima

- **Bom uso:** Pessoa → Endereco (**OneToOne**)

- **Péssimo uso:** Pessoa → Veículos (**OneToMany**) → remove veículos ao remover dono (perigoso!)

1.6.1 Regra Geral

Caso	Cascade recomendado
Relação de dependência (filho só existe com pai)	<code>CascadeType.ALL</code>
Relação independente (cada um tem vida própria)	Sem cascade

2. Boas práticas de relacionamentos JPA

- Garanta que **sempre o lado dono** é atualizado nas associações;
- Evite relações bidirecionais se não forem necessárias;
- Use **@OneToMany** **somente com mappedBy** para não criar tabela extra;
- Use `orphanRemoval = true` se quiser deletar filhos automaticamente;
- Avalie se **cascade** faz sentido (perigo de apagar dados por cascata!);

3. Consultas JPQL

3.1 Consultas Básicas

Consulta	Explicação
<code>SELECT p FROM Pessoa p</code>	Retorna todas as entidades Pessoa. (equivalente a <code>SELECT *</code> em SQL, mas retornando objetos)
<code>SELECT p.nome FROM Pessoa p</code>	Retorna somente o atributo nome de cada Pessoa (apenas coluna, não objeto).
<code>SELECT p FROM Pessoa p WHERE p.id = :id</code>	Busca uma pessoa pelo ID usando parâmetro nomeado.
<code>SELECT p FROM Pessoa p WHERE p.nome LIKE :nome</code>	Busca por nome com filtro parcial (usa <code>%</code> no parâmetro).

<code>SELECT p FROM Pessoa p ORDER BY p.nome ASC</code>	Retorna pessoas ordenadas pelo nome em ordem crescente.
<code>SELECT COUNT(p) FROM Pessoa p</code>	Retorna a quantidade de registros na tabela pessoa.
<code>SELECT DISTINCT p.cargo FROM Pessoa p</code>	Retorna cargos sem repetição.
<code>SELECT p FROM Pessoa p WHERE p.salario BETWEEN :min AND :max</code>	Filtro entre intervalo numérico.
<code>SELECT p FROM Pessoa p WHERE p.dataNascimento < :data</code>	Retorna pessoas com data anterior ao parâmetro.

3.2 Consultas Avançadas (JOIN, agregação, relacionamentos)

Consulta	Explicação
<code>SELECT v FROM Veiculo v WHERE v.pessoa.id = :id</code>	Busca veículos de uma pessoa — 1:N via chave estrangeira.
<code>SELECT m FROM Manutencao m WHERE m.veiculo.id = :id</code>	Busca manutenções de um veículo.
<code>SELECT p FROM Pessoa p JOIN p.veiculos v</code>	Faz JOIN e retorna pessoas que possuem veículos.
<code>SELECT p.nome, v.nome FROM Pessoa p JOIN p.veiculos v</code>	JOIN retornando campos de entidades relacionadas.
<code>SELECT p FROM Pessoa p LEFT JOIN p.veiculos v</code>	Retorna todas as pessoas, mesmo sem veículos (LEFT JOIN).
<code>SELECT p FROM Pessoa p WHERE p.veiculos IS EMPTY</code>	Retorna pessoas sem veículo (consultando coleção vazia).
<code>SELECT p FROM Pessoa p WHERE p.veiculos IS NOT EMPTY</code>	Retorna pessoas que possuem veículo(s).

<code>SELECT AVG(p.salario) FROM Pessoa p</code>	Média salarial (função de agregação).
<code>SELECT MAX(p.salario) FROM Pessoa p</code>	Maior salário.
<code>SELECT p.cargo, COUNT(p) FROM Pessoa p GROUP BY p.cargo</code>	Agrupa pessoas por cargo e conta quantidade em cada grupo.
<code>SELECT v FROM Veiculo v WHERE v.pessoa.cargo.nome = :cargo</code>	Navega relacionamento aninhado (Pessoa → Cargo).
<code>DELETE FROM Pessoa p WHERE p.id = :id</code>	Exclusão via JPQL (não remove relacionamentos automaticamente).
<code>UPDATE Pessoa p SET p.salario = p.salario * 1.1</code>	Atualiza todos os salários (+10%).

3.3 Consultas para evitar exclusão indevida (verificação de dependência)

Objetivo	Consulta
Verificar se pessoa tem veículos	<code>SELECT v FROM Veiculo v WHERE v.pessoa.id = :id</code>
Verificar se veículo tem manutenções	<code>SELECT m FROM Manutencao m WHERE m.veiculo.id = :id</code>
Verificar se coleção está vazia (JPQL)	<code>p.veiculos IS EMPTY</code>
Verificar se coleção tem elementos	<code>p.veiculos IS NOT EMPTY</code>

3.4 Dicas

Dica	Explicação
------	------------

JPQL consulta **objetos**, não tabelas.

Ex: `FROM Pessoa` usa nome da entidade, não o nome SQL da tabela.

Nunca use `SELECT *` em JPQL.

Sempre retorna entidades ou atributos.

JPQL não usa `INNER JOIN ON`. A condição está no relacionamento.

Toda consulta deve usar nome de atributo, não nome de coluna SQL.

